

Neural Networks and Backpropagation from Scratch

Intro

The Original Goal

The original goal came from pure curiosity:

Build the smallest possible implementation of a neural network model.

I wanted to understand how little code and machinery a working model actually needs.

At first, this sounded like a concrete engineering task.

Why the Goal Is Ill-Posed

The goal described what to build, but not what should count as success.

- A binary classifier no better than a coin flip is still a model.
- If accuracy is 80%, should we continue to 81%, 82%, or 99%?
- Quality alone ignores training time and computational cost.
- A model that reaches high accuracy only after impractically long training is not a useful result.

A well-posed goal needs constraints

Define a target metric, a resource budget, and a stopping rule.

Model, Dataset, and Loss

Neural network A function f that maps an input x to a prediction $\hat{y} = f(x)$.

Dataset Examples (x, y) : an input and the expected output.

Loss For classification, the fraction of wrong answers:

$$L(f, D) = \frac{\text{wrong answers}}{\text{all answers}}.$$

Training

Change the network to make the loss smaller.

* This error rate is useful for evaluation but is not differentiable, so training uses a loss such as cross-entropy.

Naive Algorithm: Brute-Force Selection

The function `rand_net()` returns a random neural network f .

```
best = rand_net()
for i = 1 to 1000:
    candidate = rand_net()
    if L(candidate, D) < L(best, D):
        best = candidate
```

Sample many random networks and keep the one with the lowest loss.

The problem

Every candidate starts from scratch and discards what the current best network has already learned. Among all possible networks, the chance of sampling a good one is vanishingly small.

Naive Algorithm 2: Random Local Search

The function `perturb(f, step)` returns a randomly modified network close to f .

```
best = rand_net()
step = 0.001
for i = 1 to 1000:
    candidate = perturb(best, step)
    if L(candidate, D) < L(best, D):
        best = candidate
```

Instead of replacing the whole network, change it slightly and accept only variants that reduce the loss.

The problem

A random perturbation mixes helpful and harmful changes. In a large search space it is usually poorly aligned with the best direction, so the chance of a substantial improvement remains very small.

Gradient Descent

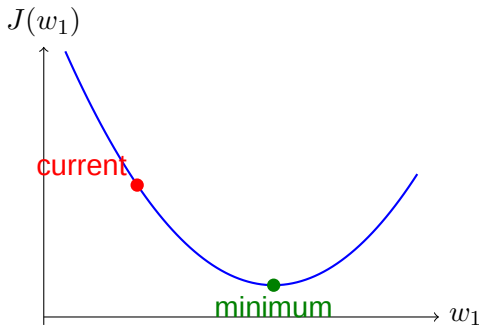
One Parameter, One Loss Curve

Consider a neural network f_{w_1} controlled by a single parameter w_1 .

For a fixed dataset D , its loss is now a function of one variable:

$$J(w_1) = L(f_{w_1}, D).$$

Training becomes a geometric problem: find the lowest point on this two-dimensional graph.



From a Limit to a Numerical Derivative

Define $J(w_1) = L(f_{w_1}, D)$. The derivative is the limit of the slope between two increasingly close points:

Exact derivative

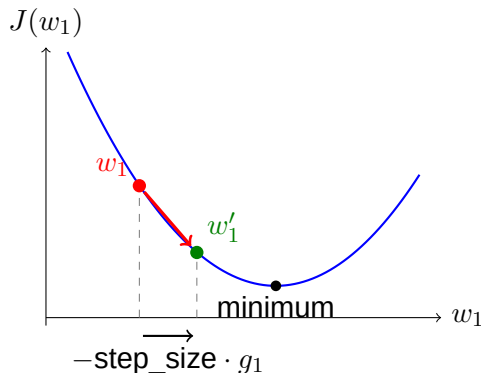
$$J'(w_1) = \lim_{h \rightarrow 0} \frac{J(w_1 + h) - J(w_1)}{h}.$$

Finite difference

$$h = 1\text{e-}5$$
$$g_1 = \frac{J(w_1 + h) - J(w_1)}{h}.$$

For a small, finite h , $g_1 \approx J'(w_1)$: it estimates how quickly the loss grows when w_1 increases.

One Gradient Descent Step



Update rule

$$\text{step_size} = 0.01$$

$$w'_1 = w_1 - \text{step_size} \cdot g_1$$

g_1 points uphill, so the minus sign moves the parameter downhill.

Recompute the derivative at the new point and repeat.

From One Parameter to Many

Let $W = (w_1, \dots, w_n)$ and collect all partial derivatives in a vector G . The network is f_W , and its objective is $J(W) = L(f_W, D)$.

```
base_loss = J(W)
G = zeros(n)
for i = 1 to n:
    W_probe = copy(W)
    W_probe[i] += h
    G[i] = (J(W_probe) - base_loss) / h

W = W - step_size * G
```

* We evaluate J independently for every parameter: once for each perturbed parameter, plus one shared evaluation of `base_loss`.

Why Numerical Gradients Do Not Scale

With a sufficiently small h , finite differences can train the network. The problem is speed.

Cost of one training step

Compute one base loss and one perturbed loss for each parameter:

$$\underbrace{N + 1}_{\text{full loss evaluations}} \implies \text{total work} = O(NC_J),$$

where C_J is the cost of evaluating $J(W) = L(f_W, D)$ once.

The bottleneck

Networks that do anything non-trivial have a huge number of parameters. One training step therefore requires a huge number of evaluations of J .

Optimizing Gradient Descent

Mini-Batch Gradient Descent

Instead of computing the gradient on the entire dataset at once, shuffle the examples and split them into mini-batches:

$$D \longrightarrow B_1, B_2, \dots, B_k.$$

Compute the gradient on one mini-batch, update the parameters, and continue with the next. Reshuffle the data before the next epoch.

Training loop

```
for batch in batches:  
    G = compute_gradient(W, batch)  
    W -= step_size * G
```

Here, `compute_gradient(W, batch)` means $\nabla_W L(f_W, \text{batch})$: the gradient of the batch loss with respect to W .

Backpropagation

Here we develop an intuitive understanding of backpropagation.

Intuition

Backpropagation computes gradients efficiently by reusing intermediate derivatives from the chain rule.

It turns one forward evaluation into gradients for all parameters with a single reverse pass through the computation graph.

Chain Rule

Recall the derivative of a function of one variable:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

For several inputs, keep the other inputs fixed:

$$f(x, y) = xy : \quad \frac{\partial f}{\partial x} = y, \quad \frac{\partial f}{\partial y} = x,$$

$$f(x, y) = x + y : \quad \frac{\partial f}{\partial x} = 1, \quad \frac{\partial f}{\partial y} = 1.$$

Definition

The chain rule computes the derivative of a composition of two or more functions from their individual derivatives. If $f(x) = g(h(x))$, then

$$f'(x) = g'(h(x))h'(x).$$

Compound Expressions with the Chain Rule

Consider a more complicated expression composed of multiple functions:

$$f(x, y, z) = (x + y)z.$$

Name the inner expression $q = x + y$. Then $f = qz$, and

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x},$$

$$\frac{\partial f}{\partial q} = z,$$

$$\frac{\partial f}{\partial x} = z \cdot 1 = z.$$

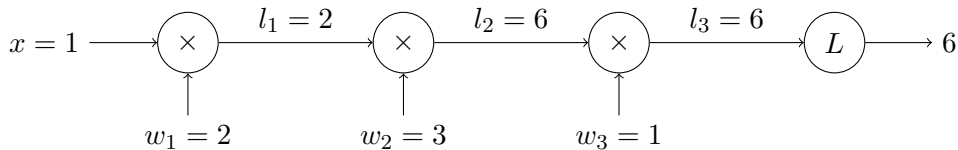
$$\frac{\partial q}{\partial x} = 1,$$

A trillion-parameter LLM, written as one formula at just 1 mm per parameter, would be about 2.6 Earth–Moon distances long.

A Computation Graph

Backpropagation applies the chain rule to a differentiable computation graph. A neural network is one particular case.

For this example, use the simplest possible loss: $L(l_3) = l_3$.



First, evaluate the graph from left to right.

Gradient for One Weight

$$g_2 = \frac{\partial L}{\partial w_2} = \frac{\partial l_2}{\partial w_2} \frac{\partial L}{\partial l_2}.$$

Known from left to right

Since $l_2 = l_1 w_2$,

$$\frac{\partial l_2}{\partial w_2} = l_1 = 2.$$

The value l_1 was stored while evaluating the graph.

Known from right to left

Since $L(l_3) = l_3$ and $l_3 = l_2 w_3$ with $w_3 = 1$,

$$\frac{\partial L}{\partial l_2} = w_3 \frac{\partial L}{\partial l_3} = 1.$$

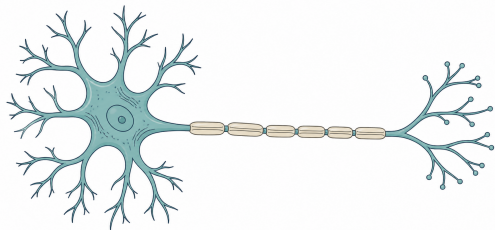
Combine both values

$$g_2 = l_1 \frac{\partial L}{\partial l_2} = 2 \cdot 1 = 2.$$

From Biological Neuron to Neural Network

From Biology to Computation

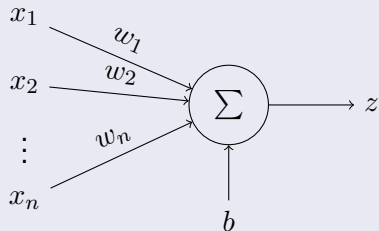
Biological neuron



Dendrites receive signals; the axon carries the neuron's output toward its terminals.

The artificial neuron is an analogy, not a detailed simulation of a living cell.

Computational model



Computes one number from numeric inputs and weights.

A Neuron as a Computation

The graph multiplies every input by its weight, adds the results, and adds a bias:

Scalar form

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b.$$

Collect the inputs and weights into vectors:

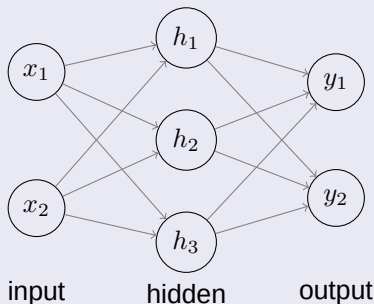
$$x = (x_1, \dots, x_n), \quad w = (w_1, \dots, w_n).$$

Vector form

$$z = w^\top x + b.$$

Same Network, Two Representations

Computation graph: $2 \rightarrow 3 \rightarrow 2$



The same network as matrices

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \underbrace{\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}}_{W_1 \ (3 \times 2)} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b_1$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \end{bmatrix}}_{W_2 \ (2 \times 3)} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} + b_2$$

Each row of a matrix contains all incoming weights of one neuron.

Without Nonlinearity, Depth Disappears

$$x \xrightarrow{W_1, b_1} h_1 \xrightarrow{W_2, b_2} h_2 \xrightarrow{W_3, b_3} y$$

$\equiv$

$$x \xrightarrow{W_*, b_*} y$$

$$\begin{aligned} y &= W_3(W_2(W_1x + b_1) + b_2) + b_3 \\ &= \underbrace{W_3W_2W_1}_{W_*}x + \underbrace{W_3W_2b_1 + W_3b_2 + b_3}_{b_*} \\ &= W_*x + b_*. \end{aligned}$$

ReLU: A Simple Nonlinearity

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}'(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases}$$

Why use ReLU?

- One `max` operation.
- Its derivative is just a binary mask.
- It works well in practice.

Other choices

- Sigmoid or tanh
- Leaky ReLU
- GELU or SiLU

One Hidden Layer in Pseudocode

A layer combines its inputs and then applies the activation function:

Python-like pseudocode

```
def hidden_layer(x, weights, biases):  
    weighted_sum = weights @ x + biases  
    return relu(weighted_sum)
```

- @ combines every input with a learned weight.
- The biases shift the result.
- ReLU keeps positive values and replaces negative values with zero.

Forward Pass

Apply the same operation to every hidden layer. Keep their outputs because the backward pass will need them:

Python-like pseudocode

```
def forward(x, weights, biases):  
    activations = [x]  
  
    for layer in range(len(weights) - 1):  
        x = weights[layer] @ x + biases[layer]  
        x = relu(x)  
        activations.append(x)  
  
    prediction = weights[-1] @ x + biases[-1]  
    return prediction, activations
```

Backward Pass

Start with how the prediction should change, then visit the layers in reverse order:

Python-like pseudocode

```
def backward(target, prediction, weights, activations):  
    d = loss_gradient(prediction, target)  
  
    for layer in reversed(range(len(weights))):  
        previous = activations[layer]  
        weight_grads[layer] += outer(d, previous)  
        bias_grads[layer] += d  
  
        if layer > 0:  
            d = weights[layer].T @ d  
            d *= relu_derivative(activations[layer])
```

Here, d says how the current layer's output should change to reduce the loss. Each iteration reuses it to continue one layer back.

One Mini-Batch Training Step

Add the gradients from the examples, average them, and update the network once:

Python-like pseudocode

```
def train(batch):  
    clear_gradients()  
  
    for x, target in batch:  
        prediction, activations = forward(x, weights, biases)  
        backward(target, prediction, weights, activations)  
  
    average_gradients(len(batch))  
    apply_gradients(step_size)
```

`apply_gradients` moves every weight and bias a small step in the direction that reduces the loss.

Thank you for your attention!

Source code

https://github.com/kvladsrc/gym/blob/main/neural_network/nn_lib/nn_lib.cc